

Distributing DBPS Shared Libraries

1. Background

The DataBatchProtectionService (DBPS) offers data protection functionality. DBPS provides C++ shared libraries (*.so files), which enable consumer projects to access the protection functionality. Currently, consumer projects can only access the shared libraries by downloading and building DBPS code. Our goal is to simplify that process.

In this document, we evaluate options and provide recommendations to accomplish this goal.

2. Available Technologies

In this section, we describe some of the technologies available to help build a solution for distributing the shared libraries

The solution space is divided into three main categories: (a) C++-specific package managers (like Conan and VCPKG) for managing C++ dependencies and builds across platforms; (b) OS-native package managers (such as APT/DEB and YUM/RPM, including OBS) for creating and distributing OS-specific packages, primarily for Linux; and (c) Artifact Repositories (like JFrog Artifactory and Cloudsmith) that host various artifact types and integrate with creation tools to present artifacts as part of OS-specific repositories.

2.1 Conan

<https://conan.io/>

Conan is a multi-platform manager for C and C++ packages. Conan does not host package binaries itself. Instead, it works with package repositories for hosting package binaries. Conan hosts “recipe” files which indicate how to build a package in different systems. Package users can download the binaries from a repository (if available); otherwise, they have the option to build a package using its recipe file.

Conan is a well maintained open source project, with permanent funding from JFrog (company which provides artifact storage and vending capabilities).

Pros:

- Well supported, industry standard for C++ packages.
- Supports multiple operating systems (Linux, Mac, Windows)
- Semi-automatic support for cross-OS, cross-CPU builds.

- Has integration with GitHub actions.

Cons:

- Requires additional solution for artifact-handling
- Requires additional tooling to be installed on package consumers
- Packages are 'project centric', rather than 'machine centric' (i.e. Conan requires additional steps to install shared libraries on a machine)

2.2 VCPKG

<https://vcpkg.io/en/>

Similar to Conan, Vcpkg is a manager for C and C++ packages. The main use case for vcpkg is hosting "recipes" on how to build package binaries. To speed up package download/building, It offers a 'binary' cache which is not populated automatically by vcpkg (but can be populated manually/via CI/CD pipelines, etc).

Pros: (almost identical to Conan)

- Well supported, industry standard for C++ packages
- Supports multiple operating systems (Linux, Mac, Windows)
- Has integration with GitHub actions

Cons: (almost identical to Conan)

- Requires additional solution for artifact-handling
- Requires additional tooling to be installed on package consumers
- Packages are 'project centric', rather than 'machine centric' (i.e. Conan requires additional steps to install shared libraries on a machine)

2.3 Open Build Service (OBS)

<https://openbuildservice.org/>

OBS is a generic system to build and distribute packages from sources. It enables the release of packages and updates for a wide range of OS and hardware architectures.

To build a package in OBS, package owners upload the source files to OBS, and then instruct OBS to build the package. OBS then hosts the built packages.

Pros

- Automatically creates and distributes artifacts for a variety of targets (e.g. OS+CPU arch combination)
- Mature and maintained.

- Supports a breadth of Linux distributions and hardware architectures
- Has integration with GitHub actions.

Cons

- For non-open source projects, must be self-hosted
- Linux-only

2.4 GitHub Actions

RPM Build: <https://github.com/marketplace/actions/rpm-build>

Deb Build: <https://github.com/marketplace/actions/build-debian-packages>

The Marketplace in GitHub Actions offers solutions for packaging code builds into OS-specific artifacts (RPM, Deb), directly as part of the CI/CD pipeline.

The result of these GitHub actions is artifacts downloadable from GitHub. Generated artifacts are not published to OS-specific repositories.

GitHub actions are executed in “runners”, which are virtual-machine-like instances that can be configured for specific operating systems and CPU architectures.

Pros

- Highly supported, well documented
- Direct integration with GitHub (reduced infrastructure maintenance)

Cons

- Requires additional solution for OS-specific repositories

2.5 Artifact Repositories

JFrog Artifactory: <https://jfrog.com/artifactory/>

Cloudsmith: <https://cloudsmith.com/product/formats>

These companies offer solutions to host artifacts of different kinds, among them, they can host artifacts (and repositories) for RPM, DEB, Conan (among others)

These Artifact repositories generally do not provide a solution for artifact creation (e.g. compiling code into a library), but they typically have integrations with solutions that do (e.g. Conan, GitHub, OBS).

An Artifact Repository (e.g. JFrog, Cloudsmith) allows to present specific artifacts as part of OS-specific repositories (e.g. RPM, DEB) which can be installed in package-consumer

machines. This allows for a simple installation/upgrade of packages (e.g. `apt-get install <some_package_with_shared_lib>`)

2.6 [Recommended] Building the libraries through code dependencies

In this solution, we will signal a dependency (in the Arrow codebase) to the DBPS repository. This will happen via `CMakeLists.txt` in the Arrow repo.

As part of the dependency definition, we will instruct Arrow to download, build, and install the DBPS shared libraries (`libdbpsLocalAgent.so`, [libdbpsRemoteAgent.so](#)) alongside the artifacts generated while building the Arrow code.

Pros:

- Simple
- No need to manage cross-OS, cross-platform versions of the libraries
- DBPS already had to be declared as a dependency in Arrow (for DBPS-related header files)

Cons:

- If libraries are updated, they must be re-built by users (as they are not in a repository).

3. Appendix

3.1 Solutions Evaluated and Discarded

3.1.1 Manual creation of artifacts (OS-specific) [not recommended]

Artifacts (*.rpm, *.dev) can be created manually and uploaded to a repository.

Pros:

- Simple solution for a reduced number of artifacts.

Cons:

- Requires package-owner setup for each OS+CPU architecture
- Manual management of artifacts
- Can get complex really quickly with the combinatorial of OS+CPU architecture targets
- Most OSs do not have publicly available repositories for user-uploaded artifacts.
Requires figuring out

3.1.2 OS-specific build systems

COPR for Fedora: <https://docs.fedoraproject.org/en-US/quick-docs/publish-rpm-on-copr/>

Ubuntu PPA/Launchpad: <https://launchpad.net/ubuntu/+ppas>

Some operating systems offer proprietary solutions for building and managing packages. These were discarded early/not evaluated in depth given their ultra-specialized nature.